

Distributed Financial Transaction Processor

Final Report

Sampath Pranay Beela

Rahul China Jagadeesha

CS6650 — Building Scalable Distributed Systems

1 Problem, Team, and Overview of Experiments

This project builds a distributed financial transaction processor that accepts transfer requests at an API layer, places work on a queue, and applies balance updates asynchronously. The core requirement is correctness under concurrency: the system must not lose money, apply a transfer twice, or produce inconsistent balances under retries or failures. The primary stakeholders are engineers building payment-style backends, operators responsible for reliability, and end users who expect transfers to remain correct under load.

Sampath Pranay Beela focuses on API design, service integration, load generation, and analysis. Rahul Chinya Jagadeesha focuses on DynamoDB schema design, worker-side concurrency control, correctness verification, and infrastructure automation.

The architecture uses a Go-based `api-svc`, a Go-based `worker-svc`, an SQS-compatible queue, and DynamoDB as the system of record. Transfers are recorded as `PENDING`, enqueued, and completed by the worker using conditional writes on account versions. Observability includes a health endpoint, transaction status records, service logs, Locust exports, and a balance reconciliation script. The planned cloud deployment extends this with CloudWatch logs and metrics.

The experimental plan has four parts: hot-account contention, worker crash recovery, optimistic versus pessimistic locking, and horizontal scaling. The main outputs are throughput, p95 latency, retry behavior, failure rate, and final balance correctness.

AI is used as a productivity tool for scaffolding, drafting, and initial code generation, but not as the final authority on correctness. Human review remains necessary for experiment design, debugging, and interpreting tradeoffs.

2 Project Plan and Recent Progress

The recent milestone focused on establishing a working local prototype before moving to cloud deployment. The repository now includes both Go services, Dockerfiles, local orchestration, queue and DynamoDB wiring, a seed script for 101 accounts, Locust load tests, and a balance verification script. The local workflow is automated through `make init`, `make smoke`, `make test-load`, `make test-load-hot`, and `make verify`. Current progress shows clean builds, a working smoke test, and zero-failure local load tests with preserved balance correctness.

The project plan is summarized in Table 1. The sequence is deliberate: local correctness first, then infrastructure, then comparative experiments, and finally synthesis.

Table 1: Project plan and ownership

Phase	Key tasks	Owner(s)	Timeline	Success criterion
Local core	Service scaffolding, DynamoDB schema, idempotency, optimistic locking, queue wiring	Both	Weeks 1–2	End-to-end transfer flow works locally and balances reconcile
Containerization and infra	Docker images, Compose, Terraform, and cloud resource setup	Rahul, Pranay	Weeks 3–4	System deploys reproducibly and passes smoke tests
Preliminary experiments	Mixed-load test, hot-account test, balance verification, metrics export	Pranay	Weeks 3–4	Stable baseline metrics captured with zero correctness violations
Comparative concurrency study	Pessimistic locking implementation and side-by-side comparison	Both	Weeks 5–6	Breakpoint between strategies identified under increasing load
Scaling study	Increase replica count and re-run correctness checks	Both	Weeks 5–6	Throughput scaling measured without breaking invariants
Final analysis	Charts, report writing, demo preparation	Both	Weeks 7–8	Final narrative and evidence are ready for submission

AI accelerates scaffolding and documentation, but its output still requires code review and measured validation.

Recent progress has also de-risked the rest of the schedule: local dependencies are stable, the shared Go module builds cleanly, and the seeded environment creates a fixed total balance of \$1,010,000 that can be checked after every test.

3 Objectives

The short-term objective is to deliver a transaction processor that enforces three invariants: idempotent transfers, globally conserved balances, and no double spending under retries. A second short-term objective is to quantify how optimistic locking behaves under both mixed traffic and concentrated contention. A third is to make the system observable enough that correctness bugs are diagnosable rather than speculative.

The long-term objective is to evolve the system into a reusable benchmark for correctness-versus-throughput trade-offs in financial write-heavy workloads. Future work includes richer failure injection, dead-letter queue analysis, request tracing, cloud-native observability, and a second coordination strategy for side-by-side comparison. More broadly, the project aims to answer when an eventually retried optimistic design is good enough and when stronger coordination is worth the cost.

4 Related Work

This project sits in the same design space as course topics on asynchronous messaging, at-least-once delivery, optimistic concurrency control, and the cost of distributed coordination. The queue-backed design decouples bursty request ingestion from balance mutation, while DynamoDB conditional writes represent the optimistic-concurrency approach the experiments are meant to evaluate.

Three Piazza projects are especially close in spirit. Tejas Shinde’s distributed live leaderboard is similar because it studies fault tolerance and horizontal scaling, but a leaderboard can tolerate brief staleness whereas this system cannot tolerate monetary inconsistency. Gideon Ntim Gyakari’s approval workflow is similar because it also uses SQS and idempotency, but it relies on PostgreSQL serializable isolation rather than DynamoDB conditional writes. Harrison Pham’s concert ticket flash sale platform is similar because correctness under intense contention is central to both systems, but ticketing prevents oversell while this project must conserve value across debit and credit operations.

5 Methodology

The implementation follows a queue-backed asynchronous model. The API service validates requests, checks whether the transaction already exists, records a **PENDING** transaction with a conditional write, and enqueues the request. The worker polls the queue, reads the sender account, verifies funds, performs a version-checked debit, credits the receiver, and marks the transaction **COMPLETED**. Version conflicts trigger up to three retries with exponential backoff. Non-recoverable conditions such as missing accounts or insufficient funds mark the transaction **FAILED**.

This design exposes the main tradeoff the project studies. Optimistic locking keeps the steady-state fast because no distributed lock service is involved, but it pushes complexity into retries when many writers target the same item. The worst-case workload is therefore a hot-account storm, while the base case is a mixed workload over many accounts.

Observability is built into the method. API responses expose transfer status, worker logs show retry behavior, Locust exports latency and throughput histories, and the verifier scans the accounts table to ensure that the seeded total balance of \$1,010,000 is preserved to within a one-cent tolerance. The experiments are incremental: the system must pass the smoke test and preserve balances before any comparative performance claims are made. AI use within the methodology is intentionally narrow; generated code and documentation are still checked against measured behavior.

6 Preliminary Results

Table 2 summarizes the current measured results from the provided load-test artifacts. The mixed run combines random transfers and hot-account transfers for five minutes with 50 users. The isolated hot-account run uses 50 users hammering a single sender account for five minutes. In both runs, the failures and exceptions CSV files are empty, indicating that Locust observed no request-level failures or worker-side exception surfacing.

Table 2: Measured preliminary results from `load-tests`

Scenario	Req.	Avg ms	p95 ms	p99 ms	Avg req/s	Peak req/s
Mixed load aggregated	81,152	4.32	10	32	270.84	280.7
Mixed random transfers	24,236	4.44	11	35	80.89	—
Mixed hot transfers	56,916	4.27	10	31	189.95	—
Hot-account only	113,950	3.86	7	28	380.38	394.7

Figure 1 visualizes the same preliminary results as time-series trends rather than summary rows. The throughput plot shows the hot-account-only run climbing to a higher steady-state request rate than the mixed run, while the p95 plot shows short warm-up spikes followed by relatively stable latency bands in both workloads.

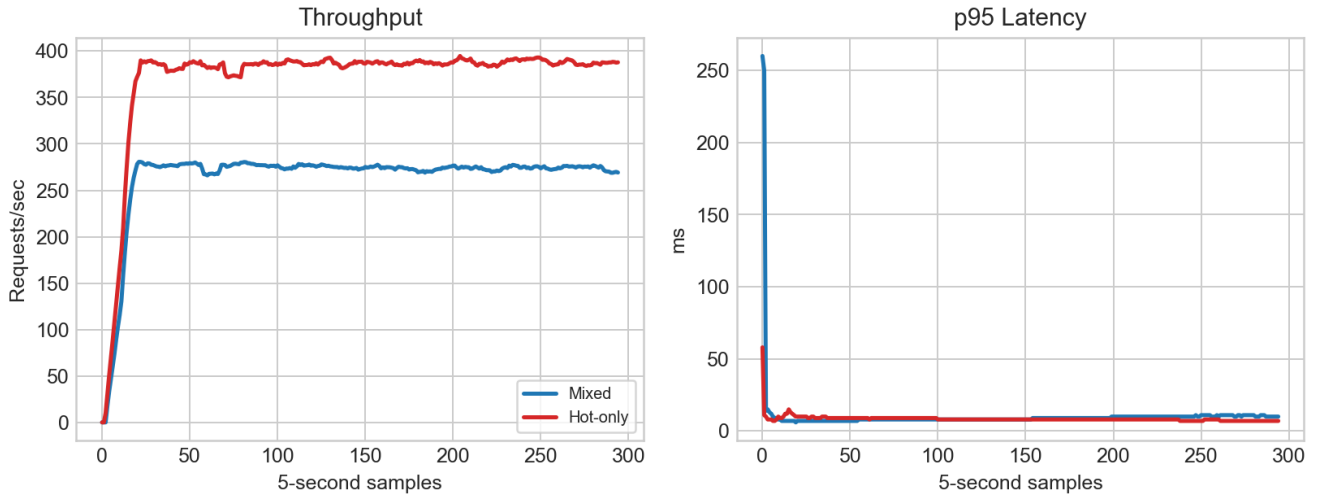


Figure 1: Throughput and p95 latency trends from the Locust history exports for mixed load and hot-account-only load.

Figures 2 and 3 add two other useful perspectives. The percentile chart makes the latency distribution comparison easier to scan than a raw table, while the volume/throughput chart separates total completed work from sustained and peak request rates.

Two findings stand out. First, the system sustained 270.84 requests/s in the mixed run and 380.38 requests/s in the hot-account-only run while keeping request-level failure rate at 0%. Second, the hot-account-only scenario did not cause the dramatic latency collapse that an optimistic-locking design might fear at this scale; its p95 latency was 7 ms, better than the 10 ms aggregated p95 in the mixed run. The history files show that the hot-account scenario reached 394.7 requests/s at peak while the mixed run peaked at 280.7 requests/s. This suggests that, at 50 concurrent users and with the current retry budget, the implementation absorbs conditional-write conflicts without surfacing instability to the client.

The mixed-run breakdown is also informative. Of the 81,152 total requests in the mixed scenario, 56,916 came from the hot-transfer workload and 24,236 came from random transfers. Despite this blend of low- and high-contention paths, the average latency remained only 4.32 ms and the p99 remained 32 ms.

The hot-account-only run is the strongest preliminary result because it tests the case that should hurt optimistic concurrency the most. It completed 113,950 requests with a median latency of 2 ms, average latency of 3.86 ms, and p99 of 28 ms while still reporting zero failures. This does not prove that optimistic locking will remain superior at higher concurrency, but it does establish that the current implementation is stable at the 50-user stress level.

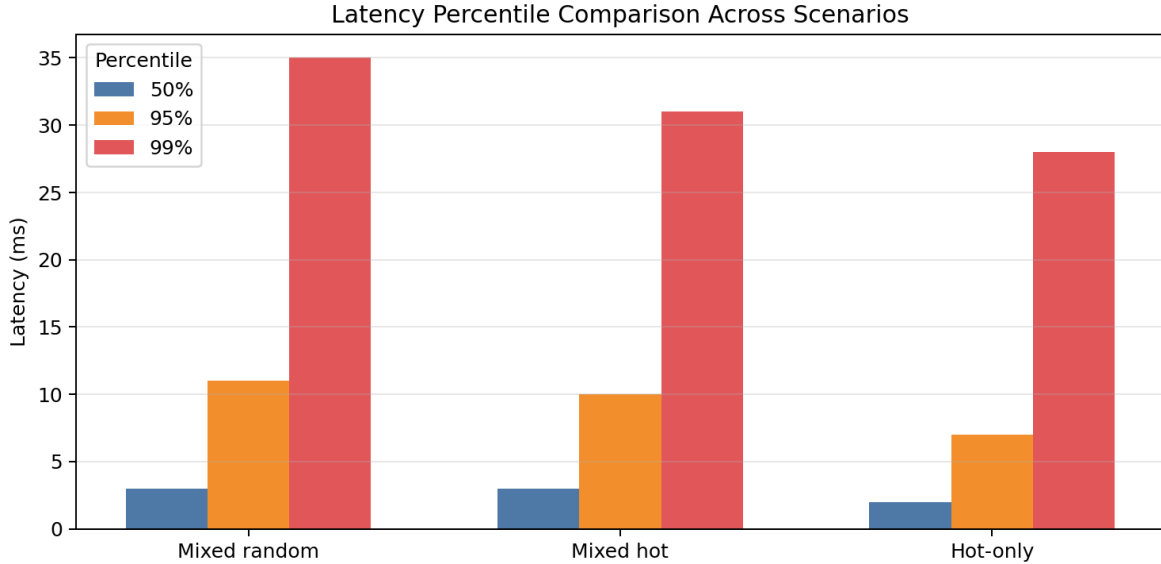


Figure 2: Latency percentile comparison across the measured scenarios using the Locust summary CSV files.

The error artifacts reinforce that interpretation. Both failure CSV files contain no failure rows, and both exception CSV files are empty. That means the local system produced zero request-level failures and zero recorded exceptions across both runs. This should not be overinterpreted as proof of general robustness, but it is useful evidence that the prototype is stable enough for larger experiments.

The current results are still preliminary because only the local mixed-load and hot-account experiments have been completed. Worker crash recovery, optimistic-versus-pessimistic comparison, and horizontal scaling remain future work.

7 Impact

This project matters because it turns a common distributed-systems claim, “eventual correctness under retries,” into something measurable for a write-heavy financial workload. Engineers can use the system and experiments to reason about when conditional writes are sufficient, when stronger coordination is worth the overhead, and how much observability is needed before trusting a queue-backed money movement pipeline.

The broader significance is practical. Many distributed applications can hide small inconsistencies behind caching or reconciliation, but a financial transaction processor cannot. A clear comparison among optimistic locking, failure recovery, and scaling can inform the design of payment services, wallet backends, or approval systems where correctness violations have direct business cost.

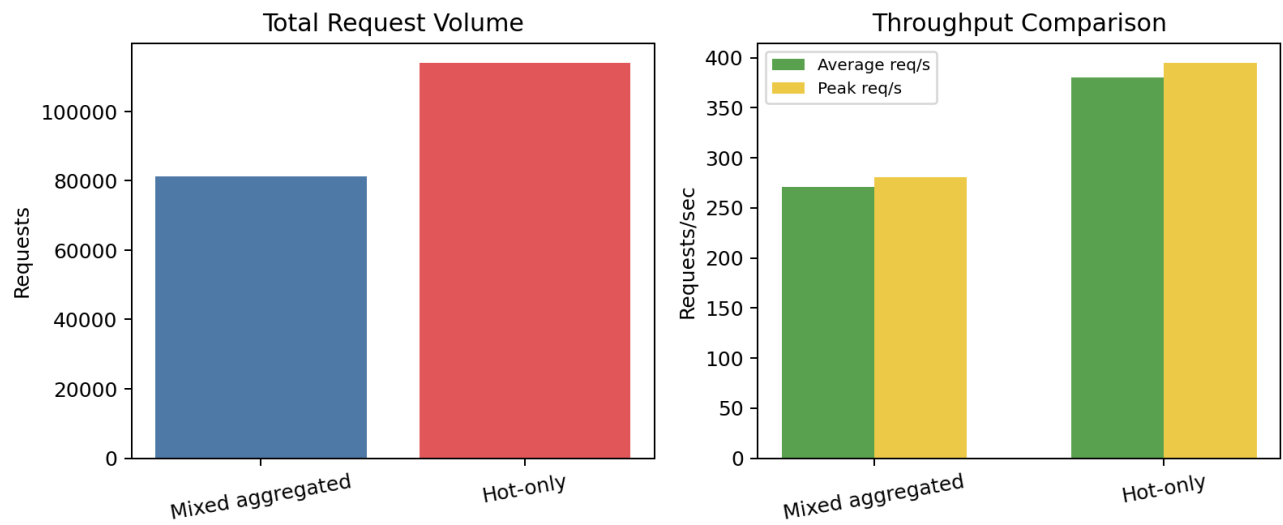


Figure 3: Request-volume and throughput comparison between the mixed aggregated run and the hot-account-only run.